

Práctica 1: Patrones de Diseño Creacionales y Estructurales

Ejercicio 3: Web Scraping con Patrón Strategy

Grado en Ingeniería Informática

Marzo 2026

1. Introducción

Este ejercicio consiste en el desarrollo de un sistema de extracción de datos (web scraping) orientado a objetos. Se ha tomado como fuente el sitio web *Scrape This Site* para recolectar estadísticas de equipos de hockey sobre hielo.

2. Descripción del Patrón Strategy

Siguiendo los requisitos de la práctica, se ha implementado el patrón de diseño **Strategy**. Este patrón estructural permite definir una familia de algoritmos (estrategias de scraping), encapsular cada uno de ellos y hacerlos intercambiables en tiempo de ejecución.

2.1. Estrategias Implementadas

Se han desarrollado las dos estrategias concretas requeridas:

- **BeautifulSoupStrategy:** Utiliza la librería `BeautifulSoup` para procesar el HTML estático obtenido mediante peticiones `requests`.
- **SeleniumStrategy:** Utiliza `Selenium WebDriver` para navegar y extraer datos del DOM, permitiendo el procesamiento de contenido dinámico si fuera necesario.

3. Implementación y Datos Extraídos

El programa solicita al usuario la estrategia deseada y procede a iterar por las **5 primeras páginas** del sitio web. Los datos extraídos para cada equipo incluyen:

- Identificación: Nombre del equipo y año.
- Rendimiento: Victorias, derrotas y derrotas en tiempo extra (OT).
- Estadísticas: Porcentaje de victorias, goles a favor/contra y diferencia de goles.

Finalmente, la información se vuelca en un archivo con formato `.csv` para su posterior análisis.

3.1. Implementación del Patrón Strategy

A continuación se muestra la implementación de la interfaz base y el contexto para nuestras estrategias de scraping:

```
1 class EstrategiaScrap(ABC):
2     @abstractmethod
3     def scrap(self, url):
4         pass
```

Listing 1: Interfaz de la Estrategia

```
1 class ContextoScrap:
2     def __init__(self, estrategia: EstrategiaScrap):
3         self._estrategia = estrategia
4
5     def set_estrategia(self, estrategia: EstrategiaScrap):
6         self._estrategia = estrategia
7
8     def realizar_scrap(self, url):
9         self._estrategia.scrap(url)
```

Listing 2: Contexto de la Estrategia

3.2. Implementación de BeautifulSoup

A continuación se muestra la implementación de la estrategia de scrap basada en BeautifulSoup:

```
1 class ScrapBeautifulSoup(EstrategiaScrap):
2     def scrap(self, url):
3         # Implementacion del scrap utilizando BeautifulSoup
4
5         equipos_info = []
6         url_paginada = f"{url}"
7
8         for i in range(1, 6):
9             print(f"Scrapeando pagina {i}...")
10
11             r = requests.get(url_paginada)
12             soup = BeautifulSoup(r.text, 'html.parser')
13
14             equipos = soup.find_all('tr', class_='team')
15
16             for equipo in equipos:
17                 nombre_equipo = equipo.find('td', class_='name').text.
18 strip()
19                 ano = equipo.find('td', class_='year').text.strip()
20                 victorias = equipo.find('td', class_='wins').text.strip
21 ()
22                 derrotas = equipo.find('td', class_='losses').text.strip
23 ()
24                 derrotas_tiempo_extra = equipo.find('td', class_='ot-
25 losses').text.strip()
26                 porcentaje_victorias = equipo.find('td', class_='pct').
27 text.strip()
28                 goles_favor = equipo.find('td', class_='gf').text.strip
29 ()
```

```

24         goles_contra = equipo.find('td', class_='ga').text.strip
25     ()
26         diferencia_goles = equipo.find('td', class_='diff').text
27     .strip()
28
29     equipos_info.append([nombre_equipo, ano, victorias,
30     derrotas, derrotas_tiempo_extra, porcentaje_victorias, goles_favor,
31     goles_contra, diferencia_goles])
32
33     url_paginada = f"{url}?page_num={i+1}"
34
35     with open('equiposBS.csv', 'w', newline='') as file:
36         writer = csv.writer(file)
37         writer.writerow(['Nombre del Equipo', 'Ano', 'Victorias'
38         , 'Derrotas', 'Derrotas en tiempo extra', 'Porcentaje de victorias',
39         'Goles a favor', 'Goles en contra', 'Diferencia de goles'])
40         writer.writerows(equipos_info)
41
42     print("Scrap completado y datos guardados en equiposBS.csv")

```

Listing 3: Implementación BeautifulSoupScrap

3.3. Implementación de Selenium

A continuación se muestra la implementación de la estrategia de scrap basada en Selenium:

```

1 class ScrapSelenium(EstrategiaScrap):
2     def scrap(self, url):
3         # Implementacion del scrap utilizando Selenium
4         print(f"Scrapeando {url} con Selenium")
5
6         chrome_options = Options()
7         driver = webdriver.Chrome(options=chrome_options)
8         driver.get(url)
9
10        equipos = []
11
12        for i in range(5): # Queremos 5 paginas
13            print(f"Analizando pagina {i+1} con Selenium...")
14            time.sleep(5)
15
16            filas = driver.find_elements(By.CLASS_NAME, 'team')
17
18            for fila in filas:
19                nombre_equipo = fila.find_element(By.CLASS_NAME, 'name')
20                .text.strip()
21                ano = fila.find_element(By.CLASS_NAME, 'year').text.
22                strip()
23                victorias = fila.find_element(By.CLASS_NAME, 'wins').
24                text.strip()
25                derrotas = fila.find_element(By.CLASS_NAME, 'losses').
26                text.strip()
27                derrotas_tiempo_extra = fila.find_element(By.CLASS_NAME,
28                'ot-losses').text.strip()
29                porcentaje_victorias = fila.find_element(By.CLASS_NAME,
30                'pct').text.strip()

```

```

25         goles_favor = fila.find_element(By.CLASS_NAME, 'gf').
text.strip()
26         goles_contra = fila.find_element(By.CLASS_NAME, 'ga').
text.strip()
27         diferencia_goles = fila.find_element(By.CLASS_NAME, '
diff').text.strip()
28
29         equipos.append([nombre_equipo, ano, victorias, derrotas,
derrotas_tiempo_extra, porcentaje_victorias, goles_favor,
goles_contra, diferencia_goles])
30
31         boton_siguiente = driver.find_element(By.XPATH, "//a[@aria-
label='Next']")
32         boton_siguiente.click()
33
34         driver.quit()
35
36         with open('equiposSelenium.csv', 'w', newline='') as file:
37             writer = csv.writer(file)
38             writer.writerow(['Nombre del Equipo', 'Ano', 'Victorias', '
Derrotas', 'Derrotas en tiempo extra', 'Porcentaje de victorias', '
Goles a favor', 'Goles en contra', 'Diferencia de goles'])
39             writer.writerows(equipos)
40
41         print("Scrap completado y datos guardados en equiposSelenium.csv
")

```

Listing 4: Implementación SeleniumScrap

3.4. Implementación del main

A continuación se muestra la implementación del main que gestiona la interacción con el usuario:

```

1 def seleccionarScrap(opcion):
2     opciones = {
3         1: ScrapBeautifulSoup(),
4         2: ScrapSelenium()
5     }
6     return opciones.get(opcion)
7
8 def main():
9     print("Seleccione la estrategia de scrap:")
10    print("1. BeautifulSoup")
11    print("2. Selenium")
12
13    opcion = int(input("Ingrese el numero de la estrategia: "))
14    estrategia = seleccionarScrap(opcion)
15
16    if estrategia is None:
17        print("Opci n no v lida.")
18        return
19
20    contexto = ContextoScrap(estrategia)
21
22    url = "https://www.scrapethissite.com/pages/forms/"
23    contexto.realizar_scrap(url)

```

```
24
25
26 if __name__ == "__main__":
27     main()
```

Listing 5: Implementación del main

4. Conclusión

La implementación cumple con los criterios de fidelidad al patrón de diseño y reutilización de métodos exigidos en la evaluación. El uso de este patrón facilita la extensión del software, permitiendo añadir nuevas librerías de scraping en el futuro sin modificar la lógica principal.